

On this page

Paddle Gateway Case Study

Learn from a real-world implementation by examining how the Paddle payment gateway was integrated into FluentCart Pro. This case study provides practical insights and patterns you can apply to your own gateway development.

Overview

The Paddle Gateway demonstrates a **complete production-ready implementation** featuring:

- **One-time payments** with full transaction lifecycle
- **Subscription payments** with recurring billing management
- **Comprehensive webhook/IPN handling** for real-time updates
- **Custom checkout experience** with UI customization
- **Multi-environment support** (test/live modes)
- **Advanced settings management** with dynamic configuration
- **Frontend integration** with JavaScript SDK
- **Error handling and logging** for production reliability
- **Security best practices** with signature verification

This case study shows how to implement **all major payment gateway features** that third-party developers typically need.

Architecture Analysis

File Structure

```
fluent-cart-pro/app/Modules/PaymentMethods/PaddleGateway/  
├─ Paddle.php                # Main gateway class  
├─ PaddleSettings.php        # Settings management  
├─ PaddleSubscriptions.php    # Subscription handling  
├─ PaddleHelper.php          # Utility functions  
├─ Processor.php             # Payment processing  
├─ Confirmations.php         # Payment confirmations  
├─ Price.php                 # Price management  
├─ Product.php               # Product handling  
├─ SubscriptionsManager.php   # Subscription lifecycle  
├─ API/  
│   └─ API.php               # API communication  
└─ Webhook/  
    └─ IPN.php               # Web hook processing
```

Main Gateway Implementation

Core Gateway Class

```
class Paddle extends AbstractPaymentGateway  
{  
    private $methodSlug = 'paddle';  
  
    public array $supportedFeatures = [  
        'payment',  
        'refund',  
        'webhook',  
        'subscriptions',  
        'custom_payment'  
    ];  
  
    public function __construct()  
    {  
        parent::__construct(  

```

php

```

        new PaddleSettings(),
        new PaddleSubscriptions()
    );

    // Register custom checkout filter
    add_filter('fluent_cart/payment_methods_with_custom_checkout_buttons',
        $methods[] = 'paddle';
        return $methods;
    });
}
}

```

Key Insights:

- Uses composition pattern with separate settings and subscription classes
- Declares all supported features upfront
- Registers custom checkout behavior during construction

Gateway Metadata

```

public function meta(): array
{
    return [
        'title' => __('Paddle', 'fluent-cart-pro'),
        'route' => 'paddle',
        'slug' => 'paddle',
        'description' => __('Pay securely with Paddle - Complete payment solution', 'fluent-cart-pro'),
        'logo' => Vite::getAssetUrl("images/payment-methods/paddle-logo.svg"),
        'icon' => Vite::getAssetUrl("images/payment-methods/paddle-logo.svg"),
        'brand_color' => '#7c3aed',
        'status' => $this->settings->get('is_active') === 'yes',
        'upcoming' => false,
        'supported_features' => $this->supportedFeatures,
        'tag' => 'beta'
    ];
}

```

php

Key Insights:

- Uses asset management system (Vite) for icons and logos
- Includes visual branding with brand colors
- Shows development status with tags
- Status reflects actual settings configuration

Settings Architecture

Hierarchical Settings Structure

```
class PaddleSettings extends BaseGatewaySettings
{
    public $methodHandler = 'fluent_cart_payment_settings_paddle';

    public static function getDefaults()
    {
        return [
            'is_active' => 'no',
            'provider' => 'api_keys',
            'live_api_key' => '',
            'live_client_token' => '',
            'live_webhook_secret' => '',
            'test_api_key' => '',
            'test_client_token' => '',
            'test_webhook_secret' => '',
            'payment_mode' => 'test',
            'tax_mode' => 'internal',
            // UI customization options
            'paddle_checkout_theme' => 'light',
            'paddle_checkout_button_text' => 'Pay with Paddle',
            'paddle_checkout_button_color' => '',
            // ... more settings
        ];
    }
}
```

php

Key Insights:

- Separates live and test credentials
- Includes UI customization options
- Uses descriptive setting keys
- Provides sensible defaults

Environment-Aware API Keys

php

```
public function getApiKey($mode = '')
{
    if (!$mode) {
        $mode = $this->getMode();
    }
    return $this->get($mode . '_api_key');
}

public function getMode()
{
    return $this->get('payment_mode');
}
```

Key Insights:

- Dynamic key resolution based on mode
- Centralized mode management
- Consistent naming patterns

Important: Payment Mode vs Order Mode

FluentCart distinguishes between two different "modes":

Payment Mode (`payment_mode` in settings): The current test/live configuration in payment gateway settings

php

```
$this->settings->getMode(); // Returns current payment mode from settings
```

Order Mode (`$order->mode`): The mode captured at checkout time and stored with the order

```
$order->mode; // Returns the mode when this specific order was placed
```

php

Usage Guidelines:

- **For API calls:** Use order mode from `$order->mode` to ensure consistency with the original transaction
- **For settings/configuration:** Use payment mode from `$this->settings->getMode()`
- **For transaction storage:** Always store `payment_mode` as `$order->mode` to preserve checkout-time context
- **For transaction URLs:** Use the stored `payment_mode` from transaction data

Complete Payment Processing Implementation

One-Time Payment Flow

```
class Processor
{
    public function handleSinglePayment(PaymentInstance $paymentInstance)
    {
        $order = $paymentInstance->order;
        $transaction = $paymentInstance->transaction;

        // Prepare payment data for external API
        $paymentData = [
            'items' => $this->prepareOrderItems($order),
            'customer' => [
                'email' => $order->email,
                'name' => $order->billing_name,
                'address' => $this->prepareAddress($order)
            ],
            'custom_data' => [
                'fct_transaction_hash' => $transaction->uuid,
```

php

```

        'fct_order_id' => $order->id
    ],
    'checkout' => [
        'url' => $this->getCheckoutUrl($order),
        'success_url' => $this->getSuccessUrl($transaction),
        'cancel_url' => $this->getCancelUrl($order)
    ]
];

// Create payment at external gateway
// Always explicitly pass the order mode for consistency
$response = API::createPaddleObject('transactions', $paymentData, $orde

if (is_wp_error($response)) {
    return [
        'success' => false,
        'message' => $response->get_error_message()
    ];
}

// Store external payment ID for webhook processing
$transaction->update([
    'vendor_charge_id' => $response['data']['id'],
    'payment_mode' => $order->mode, // Store order mode from checkout t
    'meta' => array_merge($transaction->meta ?? [], [
        'gateway_response' => $response['data']
    ])
]);

return [
    'success' => true,
    'redirect_url' => $response['data']['checkout']['url'],
    'transaction_id' => $response['data']['id']
];
}

private function prepareOrderItems($order)
{
    $items = [];
    foreach ($order->items as $item) {
        $items[] = [

```

```

        'price_id' => $item->meta['external_price_id'] ?? null,
        'quantity' => $item->quantity,
        'name' => $item->title,
        'description' => $item->description,
        'unit_price' => $item->unit_price * 100, // Convert to cents
        'total_price' => $item->total * 100
    ];
}
return $items;
}
}

```

Key Insights:

- Comprehensive order data mapping to external gateway format
- Custom data inclusion for webhook correlation
- Proper URL handling for checkout flow
- Error handling with WP_Error integration
- Transaction metadata storage for later processing

Subscription Payment Flow

```

public function handleSubscriptionPayment(PaymentInstance $paymentInstance) php
{
    $order = $paymentInstance->order;
    $subscription = $paymentInstance->subscription;
    $transaction = $paymentInstance->transaction;

    // Prepare subscription-specific data
    $subscriptionData = [
        'items' => $this->prepareSubscriptionItems($order, $subscription),
        'customer' => $this->prepareCustomerData($order),
        'billing_cycle' => $this->mapBillingCycle($subscription),
        'collection_mode' => 'automatic',
        'proration_billing_mode' => 'prorated_immediately',
        'custom_data' => [
            'fct_subscription_hash' => $subscription->uuid,

```



```

        'fct_order_id' => $order->id,
        'fct_transaction_hash' => $transaction->uuid
    ],
    'checkout' => [
        'url' => $this->getCheckoutUrl($order),
        'success_url' => $this->getSuccessUrl($transaction),
        'cancel_url' => $this->getCancelUrl($order)
    ]
];

// Add trial period if applicable
if ($subscription->trial_days > 0) {
    $subscriptionData['scheduled_change'] = [
        'action' => 'resume',
        'effective_at' => gmdate('Y-m-d\TH:i:s\Z', strtotime('+ ' . $subscri
    ];
}

// Create subscription at external gateway
// Always explicitly pass the order mode for consistency
$response = API::createPaddleObject('subscriptions', $subscriptionData, $or

if (is_wp_error($response)) {
    return [
        'success' => false,
        'message' => $response->get_error_message()
    ];
}

// Store subscription and transaction data
$subscription->update([
    'vendor_subscription_id' => $response['data']['id'],
    'status' => 'pending',
    'meta' => array_merge($subscription->meta ?? [], [
        'gateway_response' => $response['data'],
        'billing_cycle_data' => $response['data']['billing_cycle'] ?? []
    ])
]);

$transaction->update([
    'vendor_charge_id' => $response['data']['id'],

```

```

        'subscription_id' => $subscription->id,
        'payment_mode' => $order->mode, // Store order mode from checkout time
        'meta' => array_merge($transaction->meta ?? [], [
            'subscription_data' => $response['data']
        ])
    });

    return [
        'success' => true,
        'redirect_url' => $response['data']['checkout']['url'],
        'subscription_id' => $response['data']['id']
    ];
}

private function prepareSubscriptionItems($order, $subscription)
{
    $items = [];
    foreach ($order->items as $item) {
        // Map to subscription pricing structure
        $items[] = [
            'price_id' => $item->meta['recurring_price_id'] ?? $item->meta['ext
            'quantity' => $item->quantity,
            'name' => $item->title,
            'description' => $item->description,
            'billing_cycle' => [
                'interval' => $this->mapInterval($subscription->billing_interva
                'frequency' => $subscription->billing_interval_count ?? 1
            ],
            'unit_price' => $item->unit_price * 100,
            'trial_period' => $subscription->trial_days > 0 ? [
                'interval' => 'day',
                'frequency' => $subscription->trial_days
            ] : null
        ];
    }
    return $items;
}

private function mapBillingCycle($subscription)
{
    $intervalMap = [

```

```
'day' => 'day',
'week' => 'week',
'month' => 'month',
'year' => 'year'
];

return [
    'interval' => $intervalMap[$subscription->billing_interval] ?? 'month',
    'frequency' => $subscription->billing_interval_count ?? 1
];
}
```

Key Insights:

- Subscription-specific data structure with billing cycles
- Trial period handling with scheduled changes
- Separate subscription and transaction storage
- Complex item mapping for recurring pricing
- Comprehensive metadata storage for webhook processing

Payment Processing

Payment Delegation Pattern

```
public function makePaymentFromPaymentInstance(PaymentInstance $paymentInstance
{
    $order = $paymentInstance->order;

    if ($paymentInstance->subscription) {
        return (new Processor())->handleSubscriptionPayment($paymentInstance);
    }
}
```

php

```

        return (new Processor())->handleSinglePayment($paymentInstance);
    }

```

Key Insights:

- Delegates to specialized processor class
- Clean separation between single and subscription payments
- Maintains single responsibility principle

API Communication Layer

php

```

class API
{
    public static function makeRequest($endpoint, $data = [], $method = 'POST',
    {
        $settings = self::getSettings();

        if (!$mode) {
            $mode = $settings->getMode(); // Get current payment mode from sett
        }

        $apiKey = $settings->getApiKey($mode);
        if (empty($apiKey)) {
            return new \WP_Error('paddle_api_key_missing', 'API key is missing'
        }

        $baseUrl = $mode === 'test'
            ? 'https://sandbox-api.paddle.com'
            : 'https://api.paddle.com';

        $response = wp_remote_request($baseUrl . $endpoint, [
            'method' => $method,
            'headers' => [
                'Authorization' => 'Bearer ' . $apiKey,
                'Content-Type' => 'application/json',
            ],
            'body' => $method !== 'GET' ? json_encode($data) : null,
            'timeout' => 30

```

```
]);  
  
    // Error handling...  
    return $decodedBody;  
}  
}
```

Key Insights:

- Environment-aware endpoint selection
- Consistent error handling with WP_Error
- Proper timeout configuration
- Bearer token authentication
- **Fallback to settings mode** when no mode parameter provided
- **Best Practice:** Always explicitly pass the mode parameter for consistency

Comprehensive Webhook/IPN Implementation

Complete Webhook Processing Architecture

The Paddle gateway implements a sophisticated three-layer webhook processing system that handles all payment and subscription events:

```
class IPN  
{  
    private PaddleSettings $settings;  
  
    public function __construct()  
    {  
        $this->settings = new PaddleSettings();  
    }  
  
    /**  
     * Initialize all webhook event handlers  
     */  
}
```

php

```
public function init(): void
{
    // Payment lifecycle events
    add_action('fluent_cart/payments/paddle/webhook_transaction_paid', [$this->processTransactionPaid]);
    add_action('fluent_cart/payments/paddle/webhook_transaction_completed', [$this->processTransactionCompleted]);
    add_action('fluent_cart/payments/paddle/webhook_transaction_payment_failed', [$this->processTransactionPaymentFailed]);

    // Subscription lifecycle events
    add_action('fluent_cart/payments/paddle/webhook_subscription_created', [$this->processSubscriptionCreated]);
    add_action('fluent_cart/payments/paddle/webhook_subscription_activated', [$this->processSubscriptionActivated]);
    add_action('fluent_cart/payments/paddle/webhook_subscription_updated', [$this->processSubscriptionUpdated]);
    add_action('fluent_cart/payments/paddle/webhook_subscription_canceled', [$this->processSubscriptionCanceled]);
    add_action('fluent_cart/payments/paddle/webhook_subscription_paused', [$this->processSubscriptionPaused]);
    add_action('fluent_cart/payments/paddle/webhook_subscription_resumed', [$this->processSubscriptionResumed]);
    add_action('fluent_cart/payments/paddle/webhook_subscription_past_due', [$this->processSubscriptionPastDue]);
    add_action('fluent_cart/payments/paddle/webhook_subscription_payment_recurring', [$this->processSubscriptionPaymentRecurring]);

    // Refund/adjustment events
    add_action('fluent_cart/payments/paddle/webhook_adjustment_created', [$this->processAdjustmentCreated]);
    add_action('fluent_cart/payments/paddle/webhook_adjustment_updated', [$this->processAdjustmentUpdated]);
}

/**
 * Main webhook entry point and processor
 */
public function verifyAndProcess()
{
    $rawPayload = file_get_contents('php://input');
    $payload = json_decode($rawPayload, true);

    if (!$payload) {
        $this->sendResponse(400, 'Invalid JSON payload');
        return;
    }

    // Verify webhook signature if not disabled
    if (!$this->settings->isWebhookVerificationDisabled()) {
        $webhookSecret = $this->settings->getWebhookSecret();
        if (empty($webhookSecret)) {
            error_log('Paddle Webhook Failed: Webhook secret not configured');
            $this->sendResponse(500, 'Webhook secret not configured');
        }
    }
}
```

```
        return;
    }

    $verifier = new WebhookVerifier($webhookSecret);
    $paddleSignature = $_SERVER['HTTP_PADDLE_SIGNATURE'];

    if (!$verifier->verify($paddleSignature, $rawPayload)) {
        error_log('Paddle Webhook Failed: Invalid webhook signature');
        $this->sendResponse(401, 'Invalid webhook signature');
        return;
    }
}

// Get order from webhook data
$order = PaddleHelper::getOrderFromWebhookData($payload);
if (!$order) {
    $this->sendResponse(200, 'Order not found');
    return;
}

$eventType = Arr::get($payload, 'event_type');

// Validate accepted events
$acceptedEvents = [
    'transaction.paid', 'transaction.completed', 'transaction.payment_f
    'adjustment.created', 'adjustment.updated',
    'subscription.created', 'subscription.activated', 'subscription.upd
    'subscription.canceled', 'subscription.paused', 'subscription.resum
    'subscription.past_due'
];

if (!in_array($eventType, $acceptedEvents)) {
    $this->sendResponse(200, 'Event type not handled');
    return;
}

// Log webhook for debugging
do_action('fluent_cart/paddle_webhook_received', [
    'event_type' => $eventType,
    'data' => $payload,
    'raw' => $rawPayload,
```

```

        'order' => $order
    ]);

    // Process specific webhook event
    $eventTypeFormatted = str_replace('.', '_', $eventType);

    // Handle subscription renewal detection
    if ($eventTypeFormatted === 'transaction_completed' || $eventTypeFormat
        $paddleTransaction = Arr::get($payload, 'data');
        $vendorSubscriptionId = Arr::get($paddleTransaction, 'subscription_

        if ($vendorSubscriptionId) {
            $eventTypeFormatted = 'subscription_payment_received';
        }
    }

    // Fire specific event handler
    if (has_action('fluent_cart/payments/paddle/webhook_' . $eventTypeForma
        do_action('fluent_cart/payments/paddle/webhook_' . $eventTypeFormat
            'event_type' => $eventType,
            'data' => $payload,
            'raw' => $rawPayload,
            'order' => $order
        ]);

    $this->sendResponse(200, 'Webhook processed successfully');
} else {
    $this->sendResponse(200, 'No handler found for event type');
}
}
}

```

Individual Event Handlers

Payment Completion Handler

```

public function handleTransactionPaid($webhookData)
{

```

php


```

$payload = Arr::get($webhookData, 'data');
$paddleTransaction = Arr::get($payload, 'data');
$paddleTransactionId = Arr::get($paddleTransaction, 'id');

// Extract custom data for correlation
$customData = Arr::get($paddleTransaction, 'custom_data', []);
$transactionHash = Arr::get($customData, 'fct_transaction_hash');

// Find transaction by vendor ID or hash
$transactionModel = OrderTransaction::query()->where('vendor_charge_id', $p

if (!$transactionModel && $transactionHash) {
    $transactionModel = OrderTransaction::query()->where('uuid', $transacti
}

if (!$transactionModel || $transactionModel->status === Status::TRANSACTION
    return false;
}

// Use FluentCart's confirmation service
(new Confirmations()->confirmPaymentSuccessByCharge($transactionModel, [
    'vendor_charge_id' => $paddleTransactionId,
    'charge' => $paddleTransaction
]));

return true;
}

```

Subscription Management Handler

php

```

public function handleSubscriptionActivated($webhookData)
{
    $data = Arr::get($webhookData, 'data');
    $paddleSubscription = Arr::get($data, 'data');
    $order = Arr::get($webhookData, 'order');

    $customData = Arr::get($paddleSubscription, 'custom_data', []);
    $subscriptionHash = Arr::get($customData, 'fct_subscription_hash');
    $vendorSubscriptionId = Arr::get($paddleSubscription, 'id');

```

```

// Find subscription by vendor ID or hash
$subscription = Subscription::query()->where('vendor_subscription_id', $ven

if (!$subscription) {
    $subscription = Subscription::query()->where('uuid', $subscriptionHash)
}

if (!$subscription) {
    return false;
}

// Update subscription with complete data
$billCount = OrderTransaction::query()->where('subscription_id', $subscript
$updateData = [
    'vendor_subscription_id' => Arr::get($paddleSubscription, 'id'),
    'current_payment_method' => 'paddle',
    'status' => PaddleHelper::transformSubscriptionStatus(Arr::get($paddles
    'bill_count' => $billCount,
    'next_billing_date' => gmdate('Y-m-d H:i:s', strtotime(Arr::get($paddle
    'vendor_response' => json_encode($paddleSubscription)
];

$oldStatus = $subscription->status;
$subscription->update($updateData);

// Fire subscription activated event for new subscriptions
if (in_array($subscription->status, [Status::SUBSCRIPTION_ACTIVE, Status::S
    && !in_array($oldStatus, [Status::SUBSCRIPTION_ACTIVE, Status::SUBSCRIP
    && $subscription->bill_count === 0) {

    (new SubscriptionActivated($subscription, $order, $order->customer))->d
}

return true;
}

```

Refund Processing Handler

```
public function handleAdjustmentCreated($webhookData)
{
    $payload = Arr::get($webhookData, 'data');
    $paddleAdjustment = Arr::get($payload, 'data');
    $paddleAdjustmentId = Arr::get($paddleAdjustment, 'id');
    $action = Arr::get($paddleAdjustment, 'action');

    // Only handle refund adjustments
    if ($action !== 'refund') {
        return false;
    }

    $parentTransactionId = Arr::get($paddleAdjustment, 'transaction_id');
    $parentTransaction = OrderTransaction::query()->where('vendor_charge_id', $

    if (!$parentTransaction) {
        return false;
    }

    // Calculate refund amount from adjustment items
    $paddleRefundAmount = 0;
    foreach (Arr::get($paddleAdjustment, 'items', []) as $item) {
        $paddleRefundAmount += Arr::get($item, 'totals.total', 0);
    }

    $status = $this->transformAdjustmentStatus(Arr::get($paddleAdjustment, 'sta

    // Use FluentCart's refund service
    return \FluentCart\App\Services\Payments\Refund::createOrRecordRefund([
        'vendor_charge_id' => $paddleAdjustmentId,
        'payment_method' => 'paddle',
        'payment_mode' => $parentTransaction->payment_mode, // Use stored order
        'status' => $status,
        'total' => $paddleRefundAmount,
    ], $parentTransaction);
}

public function handleAdjustmentUpdated($webhookData)
{
    $payload = Arr::get($webhookData, 'data');
```

```

    $paddleAdjustment = Arr::get($payload, 'data');
    $paddleAdjustmentId = Arr::get($paddleAdjustment, 'id');

    $adjustmentTransaction = OrderTransaction::query()->where('vendor_charge_id'

    if (!$adjustmentTransaction) {
        return false;
    }

    // Update refund status
    $adjustmentTransaction->update([
        'status' => $this->transformAdjustmentStatus(Arr::get($paddleAdjustment
    ]));

    // Fire refund event if completed
    if ($adjustmentTransaction->status === Status::TRANSACTION_REFUNDED) {
        (new OrderRefund($order, $adjustmentTransaction))->dispatch();
    }

    return true;
}

```

Subscription Renewal Handler

php

```

public function handleSubscriptionPaymentReceived($webhookData)
{
    $order = Arr::get($webhookData, 'order');
    $payload = Arr::get($webhookData, 'data');
    $paddleSubscriptionId = Arr::get($payload, 'data.subscription_id');

    // Find subscription
    $subscriptionModel = Subscription::query()->where('vendor_subscription_id',

    if (!$subscriptionModel) {
        $subscriptionModel = Subscription::query()->where('parent_order_id', $o
    }

    if (!$subscriptionModel) {
        return false;
    }
}

```

```
}

// Re-sync subscription data from remote
$subscriptionModel->reSyncFromRemote();

return true;
}
```

Key Insights:

- **Complete Event Coverage:** Handles all payment, subscription, and refund events
- **Smart Event Detection:** Automatically detects subscription renewals vs new payments
- **Service Method Integration:** Uses FluentCart's service methods instead of manual hook firing
- **Robust Error Handling:** Comprehensive validation and error responses
- **Data Correlation:** Uses custom data fields to correlate webhooks with local records
- **Status Management:** Proper status transformations and updates
- **Signature Verification:** Optional but recommended security verification
- **Logging Integration:** Uses FluentCart's action system for debugging

Web Hook Implementation

Signature Verification

```
public static function verifyWebhook($payload = null, $signature = null, $mode = 'php')
{
    if ($payload === null) {
        $payload = file_get_contents('php://input');
    }

    if ($signature === null) {
        $signature = $_SERVER['HTTP_PADDLE_SIGNATURE'] ?? '';
    }
}
```

```

        return self::verifyWebhookSignature($payload, $signature, $mode);
    }

    private static function verifyWebhookSignature($payload, $signature, $mode)
    {
        $settings = new PaddleSettings();
        $secret = $settings->getWebhookSecret($mode);

        if (!$secret || !$signature) {
            return false;
        }

        // Paddle-specific signature verification logic
        // This uses Paddle's signature format and algorithm
        return hash_equals($expectedSignature, $signature);
    }

```

Key Insights:

- Flexible parameter handling (allows testing)
- Environment-aware secret selection
- Uses hash_equals for timing attack protection
- Gateway-specific signature algorithms

Event Processing

```

class IPN
{
    public function init(): void
    {
        // Register webhook event handlers
        add_action('fluent_cart/payments/paddle/webhook_transaction_paid', [$this->processTransactionPaid]);
        add_action('fluent_cart/payments/paddle/webhook_transaction_completed', [$this->processTransactionCompleted]);
        add_action('fluent_cart/payments/paddle/webhook_subscription_created', [$this->processSubscriptionCreated]);
        // ... more handlers
    }

    public function verifyAndProcess(): void
    {

```

php

```
{  
    $payload = file_get_contents('php://input');  
    $signature = $_SERVER['HTTP_PADDLE_SIGNATURE'] ?? '';  
  
    if (!API::verifyWebhook($payload, $signature)) {  
        http_response_code(401);  
        exit('Unauthorized');  
    }  
  
    $data = json_decode($payload, true);  
    $this->processWebhookEvent($data);  
  
    http_response_code(200);  
    exit('OK');  
}  
}
```

Key Insights:

- Uses WordPress action system for event handling
- Modular event processing
- Proper HTTP response codes
- Early exit after processing

Complete Frontend Integration

Multi-Script Asset Management

```
public function getEnqueueScriptSrc($hasSubscription = 'no'): array  
{  
    $paddleJsUrl = 'https://cdn.paddle.com/paddle/v2/paddle.js';  
  
    return [  
        [  
            'handle' => 'fluent-cart-checkout-sdk-paddle-js',  

```

php

```

        'src' => $paddleJsUrl,
    ],
    [
        'handle' => 'fluent-cart-paddle-checkout',
        'src' => Vite::getEnqueuePath('public/payment-methods/paddle-checko
        'deps' => ['fluent-cart-checkout-sdk-paddle-js']
    ]
];
}

public function getLocalizedData(): array
{
    return [
        'fct_paddle_data' => [
            'translations' => [
                'Paddle SDK is not loaded. Please ensure the Paddle script is i
                'Paddle client token is missing or invalid.' => __('Paddle clie
                'Pay with Paddle' => __('Pay with Paddle', 'fluent-cart-pro'),
                'Secure payment powered by Paddle' => __('Secure payment powere
                'Order creation failed' => __('Order creation failed', 'fluent-
                'Failed to create order' => __('Failed to create order', 'fluen
                'No Paddle price IDs found in order data. Please ensure Paddle
                'Error: Missing transaction ID' => __('Error: Missing transacti
                'Payment failed' => __('Payment failed', 'fluent-cart-pro'),
                'An error occurred. Please try again.' => __('An error occurred
            ]
        ]
    ];
}

```

Dynamic Order Information Handling

```

public function getOrderInfo(array $data)
{
    $cart = CartHelper::getCart();
    $checkoutHelper = new CartCheckoutHelper(true);
    $shippingChargeData = (new WebCheckoutHandler())->getShippingChargeData($ca
    $shippingCharge = Arr::get($shippingChargeData, 'charge');
}

```

php


```
$totalPrice = $checkoutHelper->getItemsAmountTotal(false) + $shippingCharge

$items = $checkoutHelper->getItems();
$hasSubscription = $this->validateSubscriptions($items);

$clientToken = $this->settings->getClientToken();
$settings = $this->settings->get();

// Prepare frontend payment arguments
$paymentArgs = [
    'client_token' => $clientToken ?: '',
    'mode' => $this->settings->getMode(), // Current payment mode from sett
    'paddle_checkout_button_text' => Arr::get($settings, 'paddle_checkout_b
    'paddle_checkout_button_color' => Arr::get($settings, 'paddle_checkout_
    'paddle_checkout_button_hover_color' => Arr::get($settings, 'paddle_che
    'paddle_checkout_button_text_color' => Arr::get($settings, 'paddle_chec
    'paddle_checkout_button_font_size' => Arr::get($settings, 'paddle_check
];

// Prepare payment details
$paymentDetails = [
    'mode' => 'payment',
    'theme' => Arr::get($settings, 'paddle_checkout_theme', 'light'),
    'amount' => Helper::toDecimalWithoutComma($totalPrice),
    'currency' => strtoupper(CurrencySettings::get('currency')),
    'locale' => (new StoreSettings())->get('locale', 'en'),
];

if ($hasSubscription) {
    $paymentDetails['mode'] = 'subscription';
}

$this->checkCurrencySupport();

wp_send_json([
    'status' => 'success',
    'payment_args' => $paymentArgs,
    'intent' => $paymentDetails,
    'has_subscription' => $hasSubscription,
    'message' => __('Order info retrieved!', 'fluent-cart-pro')
```

```
    ], 200));  
}
```

Key Insights:

- **Dynamic Asset Loading:** External SDK + custom integration scripts

Paddle Checkout JavaScript Implementation

The frontend JavaScript implementation handles the customer checkout experience:

javascript

```
// File: resources/public/payment-methods/paddle-checkout.js  
  
class PaddleCheckout {  
    constructor(form, orderHandler, response, paymentLoader) {  
        this.form = form;  
        this.orderHandler = orderHandler;  
        this.data = response;  
        this.paymentArgs = response?.payment_args || {};  
        this.intent = response?.intent || {};  
        this.paymentLoader = paymentLoader;  
        this.currentOrderData = null;  
        this.$t = this.translate.bind(this);  
    }  
  
    translate(string) {  
        const translations = window.fct_paddle_data?.translations || {};  
        return translations[string] || string;  
    }  
  
    init() {  
        const paddleButtonContainer = document.querySelector('.fluent-cart-checkout');  
        if (!paddleButtonContainer) {  
            console.error('Paddle container not found');  
            return;  
        }  
  
        // Clear any existing content  
        paddleButtonContainer.innerHTML = '';
```

```
// Initialize Paddle SDK and create button
this.initializePaddleSDK()
  .then(() => {
    this.createPaddleButton(paddleButtonContainer);
  })
  .catch((error) => {
    console.error('Paddle initialization error:', error);
    this.displayErrorMessage(paddleButtonContainer, error.message);
  });
}

async initializePaddleSDK() {
  if (typeof Paddle === 'undefined') {
    throw new Error(this.$t('Paddle SDK is not loaded. Please ensure th
  }

  const clientToken = this.paymentArgs?.client_token;
  const environment = this.paymentArgs?.mode === 'test' ? 'sandbox' : 'pr

  if (!clientToken) {
    throw new Error(this.$t('Paddle client token is missing or invalid.
  }

  Paddle.Environment.set(environment);
  Paddle.Initialize({
    token: clientToken.trim(),
    eventCallback: (data) => {
      this.handlePaddleEvent(data);
    }
  });
}

createPaddleButton(container) {
  const paddleButton = document.createElement('button');
  paddleButton.id = 'paddle-pay-button';
  paddleButton.className = 'paddle-checkout-button';
  paddleButton.innerHTML = this.$t('Pay with Paddle');
  paddleButton.style.cssText = `
    width: 100%;
    padding: 12px 24px;
    background: #1a73e8;
```

```
        color: #fff;
        border: none;
        border-radius: 6px;
        font-weight: 500;
        cursor: pointer;
    `;

    paddleButton.addEventListener('click', async (e) => {
        e.preventDefault();
        await this.handlePaddleButtonClick();
    });

    container.appendChild(paddleButton);
}

async handlePaddleButtonClick() {
    try {
        this.paymentLoader?.changeLoaderStatus('processing');

        if (typeof this.orderHandler === 'function') {
            const orderData = await this.orderHandler();
            if (!orderData) {
                throw new Error(this.$t('Failed to create order'));
            }

            // Open Paddle overlay checkout
            await this.openPaddleOverlay(orderData);
        } else {
            throw new Error(this.$t('Order handler is not properly configur
        });
    } catch (error) {
        this.paymentLoader?.changeLoaderStatus('Error: ' + error.message);
        this.displayErrorMessage(
            document.querySelector('.fluent-cart-checkout_embed_payment_con
            error.message
        );
    }
}

async openPaddleOverlay(orderData) {
    this.currentOrderData = orderData;
```

```
const paddleData = orderData?.response || orderData;
const items = paddleData?.paddle_price_ids?.map(item => ({
  priceId: item.price_id,
  quantity: parseInt(item.quantity) || 1
}));

if (!items || items.length === 0) {
  throw new Error(this.$t('No Paddle price IDs found in order data'))
}

Paddle.Checkout.open({
  settings: {
    displayMode: 'overlay',
    theme: this.intent?.theme || 'light',
    locale: this.intent?.locale || 'en'
  },
  items: items,
  customData: {
    fct_order_hash: orderData?.data?.order?.hash || '',
    fct_transaction_hash: orderData?.data?.transaction?.hash || ''
  }
});
}

handlePaddleEvent(eventData) {
  switch (eventData.name) {
    case 'checkout.completed':
      this.handlePaddlePaymentSuccess(eventData.data);
      break;
    case 'checkout.closed':
      this.handlePaddleCheckoutClosed(eventData.data);
      break;
    case 'checkout.payment.failed':
      this.handlePaddlePaymentFailed(eventData.data);
      break;
    case 'checkout.error':
      this.handlePaddleError(eventData.data);
      break;
  }
}
```

```
async handlePaddlePaymentSuccess(paddleData) {
  try {
    this.paymentLoader?.changeLoaderStatus('confirming');
    const transactionId = paddleData?.transaction_id;

    if (!transactionId) {
      this.paymentLoader?.changeLoaderStatus(this.$t('Error: Missing
        return;
    }

    const customData = paddleData.custom_data;
    const confirmResponse = await this.confirmPayment(transactionId, cu

    if (confirmResponse?.redirect_url) {
      this.paymentLoader?.changeLoaderStatus('redirecting');
      window.location.href = confirmResponse.redirect_url;
    }
  } catch (error) {
    this.paymentLoader?.changeLoaderStatus('Error: ' + error.message);
  }
}

displayErrorMessage(container, message) {
  const errorDiv = document.createElement('div');
  errorDiv.style.color = 'red';
  errorDiv.textContent = message;
  container.appendChild(errorDiv);
}

// Initialize when FluentCart triggers the event
window.addEventListener("fluent_cart_load_payments_paddle", function (e) {
  fetch(e.detail.paymentInfoUrl, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "X-WP-Nonce": e.detail.nonce,
    },
    credentials: 'include'
  }).then(async (response) => {
```

```

        response = await response.json();
        new PaddleCheckout(e.detail.form, e.detail.orderHandler, response, e.de
    }).catch(error => {
        console.error('Failed to load Paddle payment info:', error);
    });
});
});

```

- **Comprehensive Localization:** All user-facing messages translated
- **Real-time Order Processing:** Cart data processed on-demand
- **Subscription Detection:** Automatic mode switching based on cart contents
- **UI Customization:** Extensive theming and styling options
- **Currency Validation:** Pre-flight currency support checking

Advanced Production Features

Comprehensive Settings with Dynamic Instructions

php

```

public function fields(): array
{
    $webhookInstructions = $this->getWebhookInstructions();

    $testSchema = [
        'test_api_key' => array(
            'value' => '',
            'label' => __('Sandbox API Key', 'fluent-cart-pro'),
            'type' => 'password',
            'placeholder' => __('Your sandbox API key', 'fluent-cart-pro'),
            'help_text' => __('Get your API key from Paddle Dashboard > Develop
        ),
        'test_client_token' => array(
            'value' => '',
            'label' => __('Sandbox Client Token / Public Key', 'fluent-cart-pro
            'type' => 'text',
            'placeholder' => __('Your sandbox client token', 'fluent-cart-pro')
            'help_text' => __('Optional: Used for frontend checkout integration

```

```

    ),
    'test_webhook_secret' => array(
        'value' => '',
        'label' => __('Sandbox Webhook Secret', 'fluent-cart-pro'),
        'type' => 'password',
        'placeholder' => __('Your sandbox webhook secret', 'fluent-cart-pro'),
        'help_text' => __('Used to verify webhook signatures', 'fluent-cart-pro'),
    ),
    'test_webhook_desc' => array(
        'value' => $webhookInstructions,
        'label' => __('Webhook Configuration', 'fluent-cart-pro'),
        'type' => 'html_attr'
    ),
];

// Live schema similar structure...

return array(
    'notice' => [
        'value' => $this->renderStoreModeNotice(),
        'label' => __('Store Mode notice', 'fluent-cart-pro'),
        'type' => 'notice'
    ],
    'beta_notice' => [
        'value' => '<p class="text-gray-500">Paddle payment gateway is currently in beta mode.',
        'label' => __('Beta Notice', 'fluent-cart-pro'),
        'type' => 'html_attr'
    ],
    'payment_mode' => [
        'type' => 'tabs',
        'schema' => [
            [
                'type' => 'tab',
                'label' => __('Live credentials', 'fluent-cart-pro'),
                'value' => 'live',
                'schema' => $liveSchema
            ],
            [
                'type' => 'tab',
                'label' => __('Test credentials', 'fluent-cart-pro'),
                'value' => 'test',
            ]
        ]
    ]
);

```



```

        'schema' => $testSchema
    ]
]
],
'paddle_checkout_theme' => [
    'value' => 'light',
    'label' => __('Paddle Checkout Theme', 'fluent-cart-pro'),
    'type' => 'select',
    'options' => [
        'light' => ['label' => __('Light', 'fluent-cart-pro'), 'value'
        'dark' => ['label' => __('Dark', 'fluent-cart-pro'), 'value' =>
    ],
    'tooltip' => __('Theme to use for Paddle checkout modal', 'fluent-c
],
'paddle_checkout_button_text' => [
    'value' => __('Pay with Paddle', 'fluent-cart-pro'),
    'label' => __('Paddle Checkout Button Text', 'fluent-cart-pro'),
    'type' => 'text',
    'placeholder' => __('Pay with Paddle', 'fluent-cart-pro'),
    'tooltip' => __('Text to display on the Paddle checkout button', 'f
],
'paddle_checkout_button_color' => [
    'value' => '',
    'label' => __('Paddle Checkout Button Color', 'fluent-cart-pro'),
    'type' => 'color',
    'tooltip' => __('Color of the Paddle checkout button', 'fluent-cart
],
'disable_webhook_verification' => [
    'value' => 'no',
    'label' => __('Disable Webhook Verification', 'fluent-cart-pro'),
    'type' => 'checkbox',
    'tooltip' => __('Only disable this for testing purposes. Keep enabl
]
);
}

public function getWebhookInstructions()
{
    $webhook_url = $this->getWebhookUrl();
    $eventsHtml = $this->getEventshtml();

```

```
// Dynamic HTML instructions based on configuration state
$instructionsHtml =
    '<div class="paddle-webhook-instructions" style="padding:12px 0;">'
    . '<p><strong>' . esc_html__( 'Webhook URL:', 'fluent-cart-pro') . '</st'
    . '<code class="copyable-content" data-copy="' . esc_attr($webhook_url)
    . '<p>' . esc_html__( 'You should configure your Paddle webhooks to get
    . '<p>' . esc_html__( 'Select the following events:', 'fluent-cart-pro')
    . '<p style="display:flex; align-items:center; flex-wrap:wrap; gap:8px
    . '</div>';

return $instructionsHtml;
}
```

Multi-Environment Support with Security

```
public static function beforeSettingsUpdate($data, $oldSettings): array    php
{
    $mode = Arr::get($data, 'payment_mode', 'live');
    $apiKeyField = $mode . '_api_key';

    // Encrypt sensitive data before storage
    $data[$apiKeyField] = Helper::encryptKey($data[$apiKeyField]);

    return $data;
}

public static function validateSettings($data): array
{
    $mode = Arr::get($data, 'payment_mode', 'live');
    $apiKey = Arr::get($data, $mode . '_api_key');

    if (empty($apiKey)) {
        return [
            'status' => 'failed',
            'message' => __('API key is required.', 'fluent-cart-pro')
        ];
    }
}
```

```
// Could add live API connectivity test here
// $testResponse = API::testConnection($apiKey, $mode);

return [
    'status' => 'success',
    'message' => __('Paddle gateway credentials verified successfully!', 'f
];
}
```

Currency and Transaction URL Management

php

```
public function isCurrencySupported(): bool
{
    $supportedCurrencies = [
        'USD', 'EUR', 'GBP', 'JPY', 'AUD', 'CAD', 'CHF', 'HKD', 'SGD', 'SEK',
        'ARS', 'BRL', 'CNY', 'COP', 'CZK', 'DKK', 'HUF', 'ILS', 'INR', 'KRW',
        'MXN', 'NOK', 'NZD', 'PLN', 'RUB', 'THB', 'TRY', 'TWD', 'UAH', 'VND', '
    ];

    return in_array(strtoupper(CurrencySettings::get('currency')), $supportedCu
}

public function getTransactionUrl($url, $data)
{
    if (Arr::get($data, 'transaction_type') === 'refund') {
        $parentTransaction = OrderTransaction::query()->where('id', Arr::get($d
        $data['vendor_charge_id'] = $parentTransaction->vendor_charge_id;
    }

    // Use payment_mode from transaction data (stored order mode at checkout ti
    if (Arr::get($data, 'payment_mode') === 'test') {
        return 'https://sandbox-vendors.paddle.com/transactions-v2/' . Arr::get
    }

    return 'https://vendors.paddle.com/transactions-v2/' . Arr::get($data, 'ven
}

public function getSubscriptionUrl($url, $data)
```

```

{
    // Use payment_mode from subscription data (stored order mode at checkout t
    if (Arr::get($data, 'payment_mode') === 'test') {
        return 'https://sandbox-vendors.paddle.com/subscriptions-v2/' . Arr::ge
    }

    return 'https://vendors.paddle.com/subscriptions-v2/' . Arr::get($data, 've
}

```

Success and Cancel URL Handling

php

```

public function getSuccessUrl($transaction, $args = [])
{
    $paymentHelper = new PaymentHelper($this->getMeta('route'));
    return $paymentHelper->successUrl($transaction->uuid, $args);
}

public static function getCancelUrl(): string
{
    $checkoutPage = (new StoreSettings())->getCheckoutPage();
    $cartHash = $_GET['fct_cart_hash'] ?? '';

    if ($cartHash) {
        return add_query_arg([
            'fct_cart_hash' => $cartHash
        ], $checkoutPage);
    }

    return $checkoutPage;
}

```

Key Insights:

- **Dynamic Configuration:** Settings UI changes based on configuration state
- **Security-First Approach:** API key encryption and secure storage
- **Environment Awareness:** All URLs and endpoints environment-specific
- **Comprehensive Validation:** Settings validation with optional API testing

- **User Experience:** Helpful instructions and copyable content
 - **Production-Ready:** Extensive customization options for branding
 - **URL Management:** Proper success/cancel URL handling with cart state preservation
-

Registration Pattern

```
public static function register()  
{  
    fluent_cart_api()->registerCustomPaymentMethod('paddle', new self());  
}
```

php

Key Insights:

- Simple static registration method
 - Self-instantiation pattern
 - Called from bootstrap file
-

Comprehensive Lessons Learned

1. Complete Payment Lifecycle Management

- **One-time Payments:** Full transaction flow from creation to confirmation
- **Subscription Payments:** Complex billing cycles, trials, and renewal handling
- **Refund Processing:** Comprehensive adjustment and refund management
- **Status Synchronization:** Real-time status updates via webhooks

2. Three-Layer Webhook Architecture

- **Entry Point:** Single `handleIPN()` method for all webhook traffic
- **Central Processor:** `verifyAndProcess()` with signature verification and routing
- **Event Handlers:** Specific handlers for each event type using FluentCart services

- **Error Handling:** Comprehensive HTTP response codes and logging

3. Service Method Integration

- **StatusHelper:** Automatic order status sync and hook firing
- **Refund Service:** Complete refund transaction management
- **SubscriptionService:** Subscription lifecycle and renewal processing
- **Confirmation Service:** Payment confirmation with complex data handling

4. Production-Ready Security

- **Signature Verification:** Optional but recommended webhook signature validation
- **API Key Encryption:** Secure storage of sensitive credentials
- **Environment Separation:** Complete test/live mode isolation
- **Input Validation:** Comprehensive data validation and sanitization

5. Advanced Settings Architecture

- **Dynamic Configuration:** UI that changes based on current settings
- **Tabbed Interface:** Clean separation of test/live credentials
- **Visual Customization:** Extensive theming and branding options
- **Helpful Instructions:** Dynamic webhook setup guidance

6. Frontend Integration Excellence

- **Multi-Script Loading:** External SDK + custom integration scripts
- **Comprehensive Localization:** All user-facing messages translated
- **Real-time Processing:** Dynamic order and cart processing
- **UI Customization:** Extensive checkout appearance options

7. Subscription Management Complexity

- **Billing Cycle Mapping:** Complex interval and frequency handling
- **Trial Period Support:** Scheduled activation after trial periods
- **Renewal Detection:** Smart detection of subscription vs one-time payments

- **Status Transformation:** Gateway-specific to FluentCart status mapping

8. Data Architecture Patterns

- **Custom Data Fields:** Webhook correlation via custom metadata
- **Comprehensive Storage:** Full gateway response storage for debugging
- **Status Mapping:** Bidirectional status transformation
- **Transaction Relationships:** Parent-child relationships for refunds

Application to Your Gateway Development

Essential Implementation Patterns

Gateway Class Structure

```
// Complete feature declaration
public array $supportedFeatures = ['payment', 'refund', 'webhook', 'subscription'];

// Composition over inheritance
public function __construct() {
    parent::__construct(new YourSettings(), new YourSubscriptions());
}
```

php

Payment Processing Delegation

```
// Clean separation of concerns
public function makePaymentFromPaymentInstance(PaymentInstance $instance) {
    if ($instance->subscription) {
        return (new Processor())->handleSubscriptionPayment($instance);
    }
    return (new Processor())->handleSinglePayment($instance);
}
```

php

Webhook Architecture Implementation

php

```
// Three-layer pattern
public function handleIPN() { $this->verifyAndProcess(); }
private function verifyAndProcess() { /* verification + routing */ }
public function handleSpecificEvent($data) { /* use service methods */ }
```

Service Method Usage

php

```
// Use FluentCart services instead of manual hooks
(new StatusHelper($order))->syncOrderStatuses($transaction);
Refund::createOrRecordRefund([...], $parentTransaction);
SubscriptionService::recordManualRenewal($subscription, $transaction, $args);
```



Environment-Aware Configuration

php

```
// Dynamic configuration based on mode
public function getApiKey($mode = '') {
    if (!$mode) $mode = $this->getMode(); // Gets payment mode from settings
    return $this->get($mode . '_api_key');
}

// API calls should use order mode for consistency
API::createTransaction($data, $order->mode); // Use order's mode
```

Mode Management Best Practices

php

```
// ✅ Correct: Store order mode with transaction
$transaction->update(['payment_mode' => $order->mode]);

// ✅ Correct: Always explicitly pass mode to API calls
API::createTransaction($data, $order->mode);
API::refundTransaction($transactionId, $order->mode);

// ✅ Correct: Use payment mode for settings
$apiKey = $this->settings->getApiKey($this->settings->getMode());

// ⚠️ Works but not recommended: Relying on fallback
```



```
API::createTransaction($data); // Falls back to settings mode

// ❌ Wrong: Don't mix up the modes
$transaction->update(['payment_mode' => $this->settings->getMode()]);
```

Critical Success Factors

Use FluentCart's Service Layer: Never manually fire hooks - use service methods

Implement Complete Webhook Handling: Cover all payment lifecycle events

Understand Mode Management: Use `$order->mode` for API calls, `$this->settings->getMode()` for configuration

Always Pass Mode Explicitly: Don't rely on fallbacks - explicitly pass mode to all API calls

Plan for Subscriptions: Even if not initial scope, design architecture to support them

Security First: Implement signature verification and secure credential storage

Environment Awareness: Support test/live modes throughout the entire implementation

Comprehensive Error Handling: Proper HTTP responses, logging, and user feedback

Frontend Integration: Plan for JavaScript SDK integration and UI customization

Production Considerations: Currency support, URL management, transaction linking

Architecture Decisions Framework

- **Composition over Inheritance:** Separate settings, subscriptions, and processing classes
- **Service Layer Integration:** Use FluentCart's built-in services for all core operations
- **Event-Driven Webhooks:** Use WordPress actions for flexible webhook event handling
- **Environment Separation:** Complete isolation between test and live configurations
- **Security by Default:** Implement all security measures from the start
- **Extensible Design:** Plan for future features like subscriptions and advanced options

This comprehensive case study demonstrates that a complete payment gateway implementation requires careful attention to **payment lifecycles**, **webhook architecture**, **service integration**, **security**, and **user experience** - all working together in a cohesive, production-ready system.

Application to Your Gateway

Code Patterns to Adopt

Settings Structure

```
// Environment-aware getters
public function getApiKey($mode = '') {
    if (!$mode) $mode = $this->getMode();
    return $this->get($mode . '_api_key');
}
```

php

Error Handling

```
// Consistent error responses
if (is_wp_error($response)) {
    return new \WP_Error('gateway_error', $response->get_error_message());
}
```

php

Web Hook Verification

```
// Always verify webhooks
if (!$this->verifyWebhookSignature($payload, $signature)) {
    http_response_code(401);
    exit('Unauthorized');
}
```

php

Feature Detection

```
// Declare supported features clearly
public array $supportedFeatures = ['payment', 'refund', 'subscriptions', 'webho
```

php

Architecture Decisions

Use composition over inheritance for complex functionality

Separate API communication into dedicated classes

Implement proper validation at the settings level

Plan for localization from the start

Design for testability with dependency injection

Next: [Get Started](#) integrating custom payment gateways with FluentCart.

FluentCart developer documentation

Copyright © 2025 FluentCart